

CPRE 281 Project

Date: 4/30/2024

Ryan Aun Shern Chin (Section 7), Alisala Mwamba (Section 4)

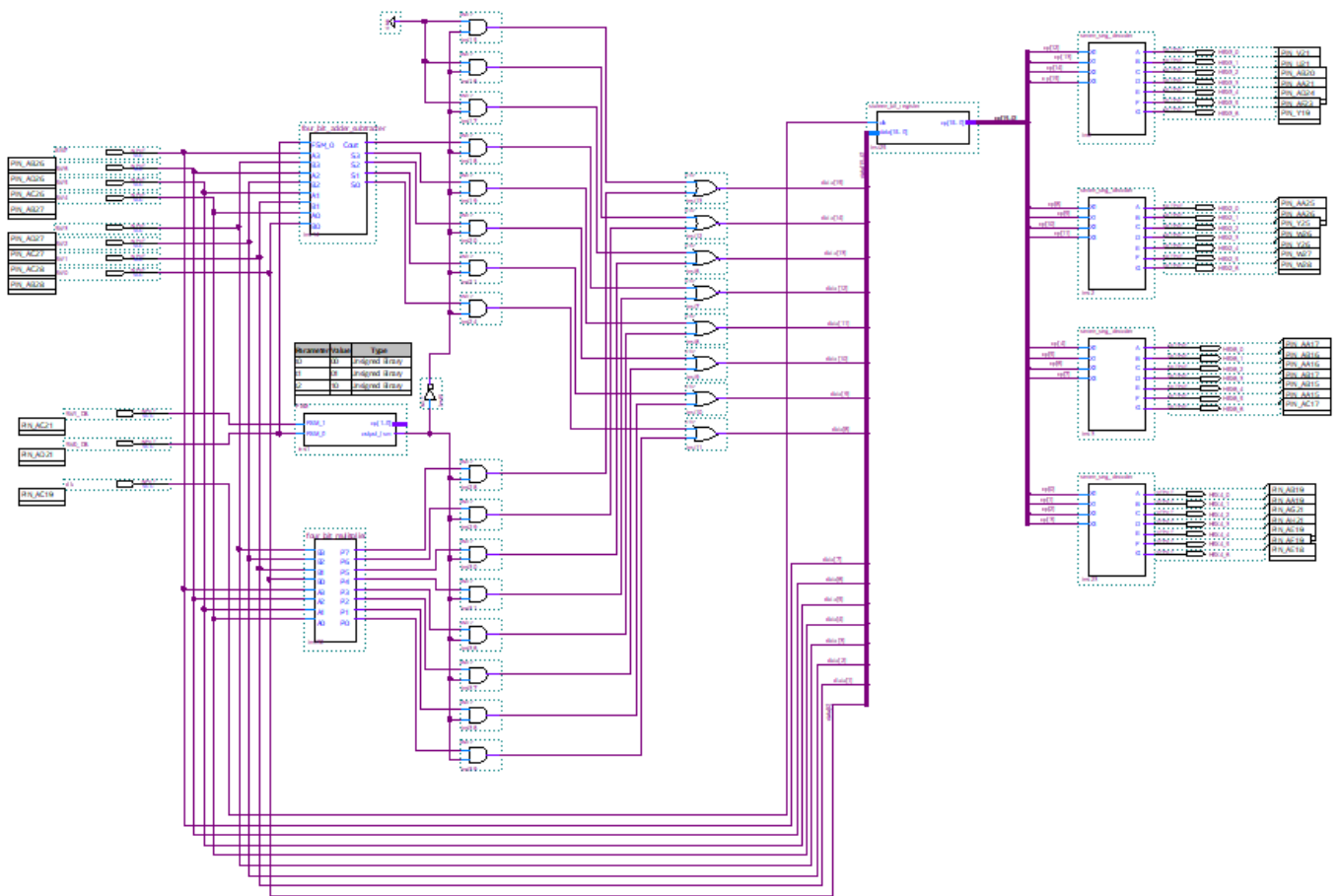
Student ID: 673329715, 510262798,

Final Project Report

We decided to work on a hexadecimal calculator that performs the functions of basic calculations such as addition, subtraction, and multiplication between two hexadecimal numbers. This report details all the workings and derivations required to build the circuit and Verilog modules contained in the project. It contains a general overview of every top-level module, followed by a multilevel description of the inner workings of each module that explains every submodule involved.

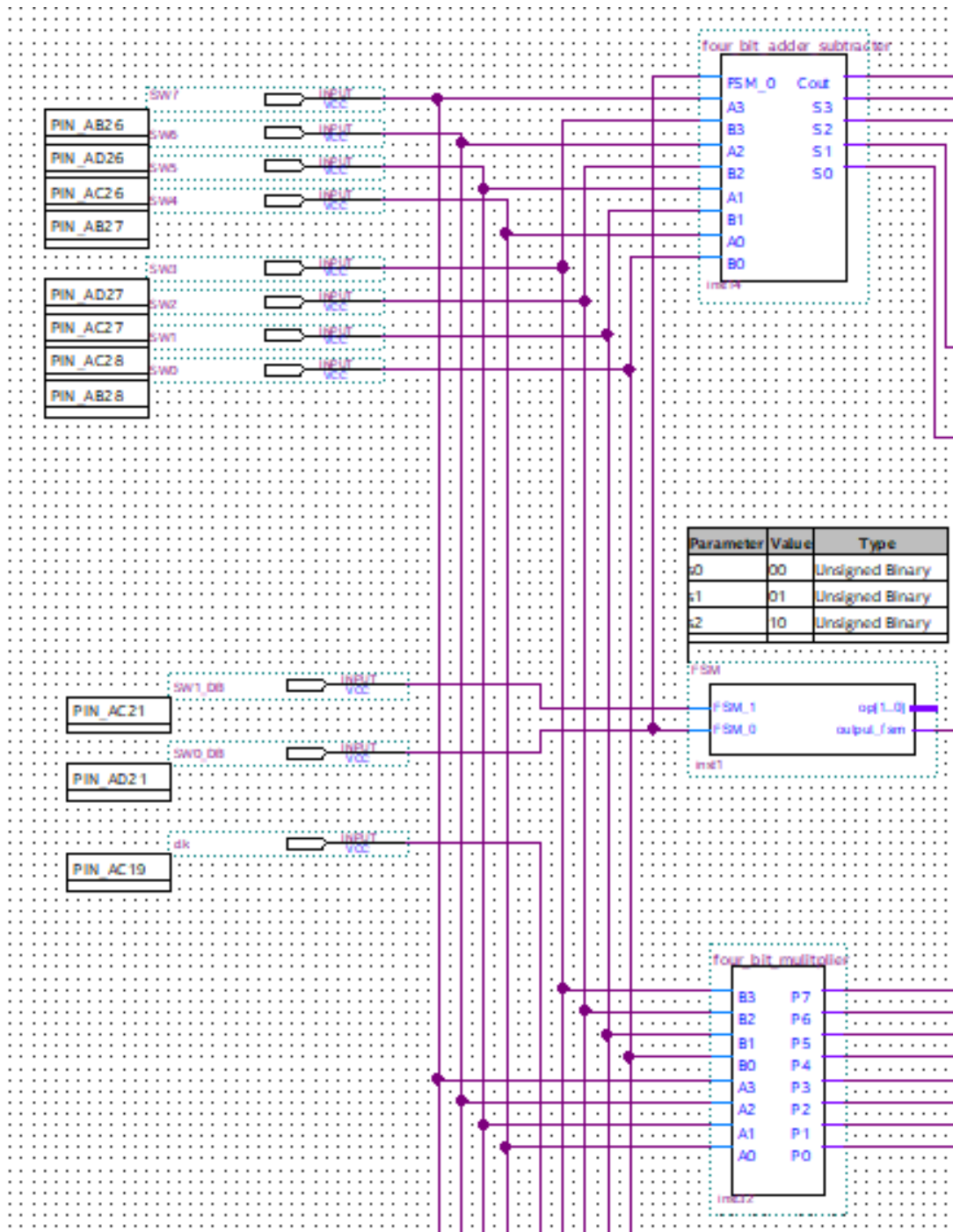
Although most pictures are given in this report, it is advised to look through the design file for each module instead since the pictures included could be a little blurry.

Top Level Diagram (*final_calculator.bdf/qpif*)



Part A

Clipping from the Top Level Diagram that represents Part A



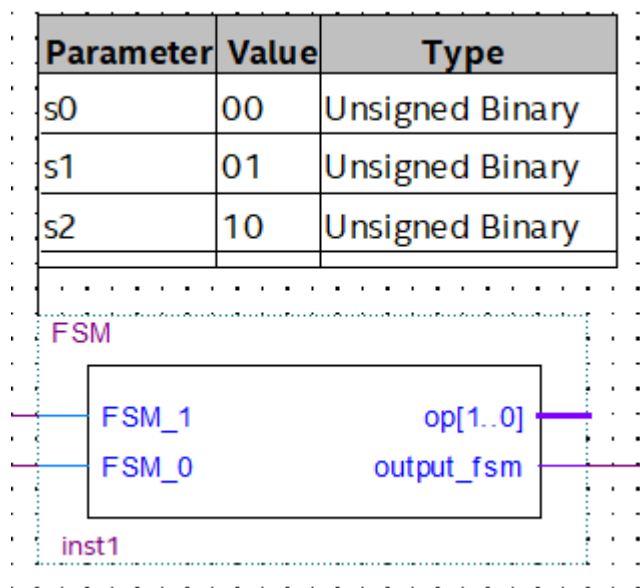
Inputs

Since the overview of the entire schematic displayed above is too large to be dissected and analyzed with the naked eye, we will differentiate our project into three parts, namely parts A, B, and C. For part A, we will first declare what inputs we will be utilizing. As we are creating a calculator that calculates the addition, subtraction, and multiplication of two numbers in hexadecimal values, we can represent each hexadecimal number that would be operated on into 4-bit values represented by 4 slide switches from **SW7** to **SW0**, utilizing four slide switches each for the two hexadecimal numbers. Therefore assigning **SW7** to **SW4** for the first hexadecimal number, **SW3** to **SW0** to the second hexadecimal number.

FSM

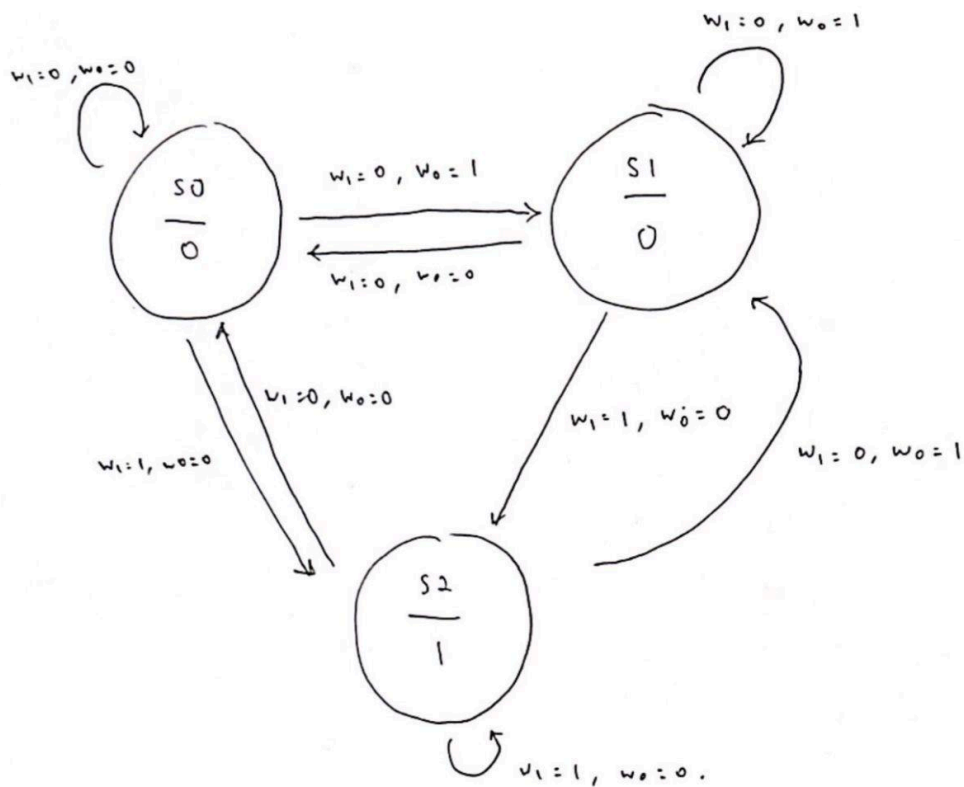
Moving on to the inputs for the Finite State Machine, represented as the FSM symbol above, we would be using two inputs from **SW1_DB** and **SW0_DB** into **FSM_1** and **FSM_0**. These two inputs represent the select lines **w1** and **w0** respectively, which would be used to determine the next state of the FSM. For example, there could only be three states the FSM could reach: S0, S1, and S2, where S0 represents addition, S1 represents subtraction, and lastly, S2 represents multiplication. We could visualize S0 being ($w1w0 = 00$), S1 being ($w1w0 = 01$) and S2 being ($w1w0 = 10$). The output of this FSM named **output_fsm** would only be 1 when the state of S2 is reached, which is later used to determine which functionality of the calculator is being triggered via the usage of a NOT gate.

Image of the FSM Symbol



We will now begin explaining the design process that went into creating this FSM symbol in the following pages to come.

Below is a state diagram that perfectly illustrates the explanation given above.



$w_1 = \text{FSM}-1$

$w_0 = \text{FSM}-0$

$z = \text{output_FSM}$

↙ This output determines whether the circuit should do add/sub or multiplication function, later in the logic.

This could be further converted into a state table and a state-assigned table, which are given below.

Present State.	(w ₁) FSM-1	(w ₀) FSM-0	Next State	output - fsm.
S ₀	0	0	S ₀	0
S ₀	0	1	S ₁	0
S ₀	1	0	S ₂	1
S ₀	1	1	X	X
S ₁	0	0	S ₀	0
S ₁	0	1	S ₁	0
S ₁	1	0	S ₂	1
S ₁	1	1	X	X
S ₂	0	0	S ₀	0
S ₂	0	1	S ₁	0
S ₂	1	0	S ₂	1
S ₂	1	1	X	X

State Assigned Table:

Present State		(w ₁)	(w ₀)	Next State		output - fsm.
y ₁	y ₀	Fsm-1	Fsm-0	op[1]	op[0]	
0	0	0	0	0	0	0
0	0	0	1	0	1	0
0	0	1	0	1	0	1
0	0	1	1	x		x
0	1	0	0	0	0	0
0	1	0	1	0	1	0
0	1	1	0	1	0	1
0	1	1	1	x		x
1	0	0	0	0	0	0
1	0	0	1	0	1	0
1	0	1	0	1	0	1
1	0	1	1	x		x

From this, we could derive the expression for the FSM using K-maps and implement the FSM circuit, but we thought of using Verilog instead since it is more efficient and the information we required to do so is already clearly defined in the state assigned table.

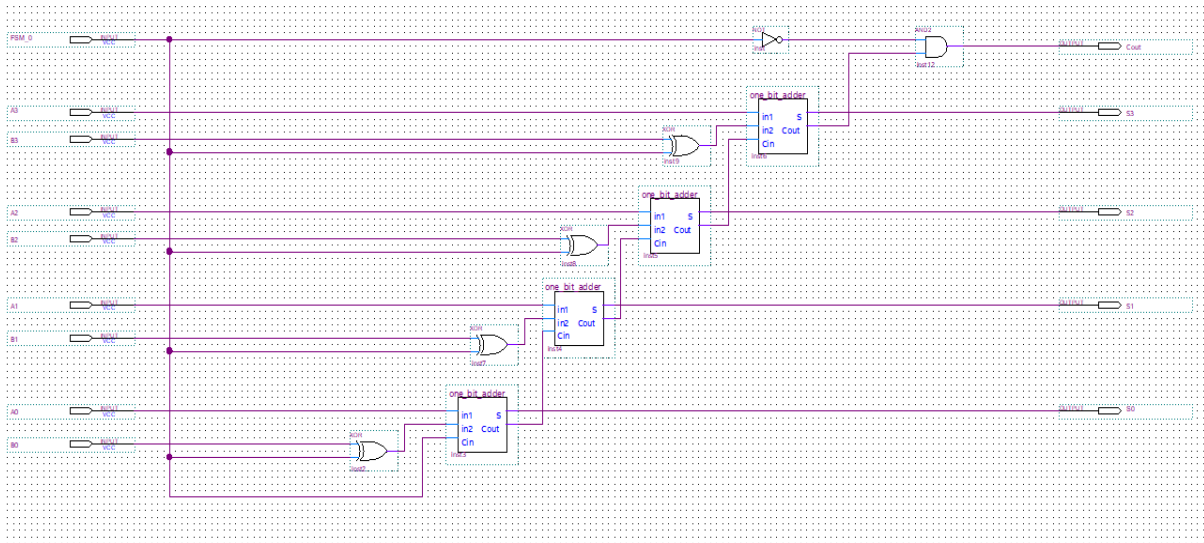
Below would be the Verilog code for the FSM:

```
1 module FSM(  
2     input wire FSM_1,  
3     input wire FSM_0,  
4     output reg [1:0] op,  
5     output reg output_fsm  
6 );  
7  
8     parameter s0 = 2'b00;  
9     parameter s1 = 2'b01;  
10    parameter s2 = 2'b10;  
11  
12    reg [1:0] state, nextstate;  
13  
14    // defining the states  
15  
16    always @(posedge FSM_0 or posedge FSM_1) begin  
17        case({FSM_1, FSM_0})  
18            2'b00: state <= s0;  
19            2'b01: state <= s1;  
20            2'b10: state <= s2;  
21            default: state <= s0;  
22        endcase;  
23    end  
24  
25    // condition of the states  
26  
27    always @* begin  
28        case(state)  
29            s0: begin  
30                op = 2'b00;  
31                output_fsm = 1'b0;  
32            end  
33            s1: begin  
34                op = 2'b01;  
35                output_fsm = 1'b0;  
36            end  
37            s2: begin  
38                op = 2'b10;  
39                output_fsm = 1'b1;  
40            end  
41            default: begin  
42                op = 2'b00;  
43                output_fsm = 1'b0;  
44            end  
45        endcase;  
46    end  
47  
48 endmodule
```

We first declared the supposed inputs and outputs of the FSM, which is followed by the declaration of state parameters. These parameters would be defined, giving them the functionality of representing each state we would be utilizing in our finite state machine. This is then followed by declaring the conditional upon reaching the states that we declared where the next state is represented by the **op** variable and the output **z** is represented by **output_fsm**.

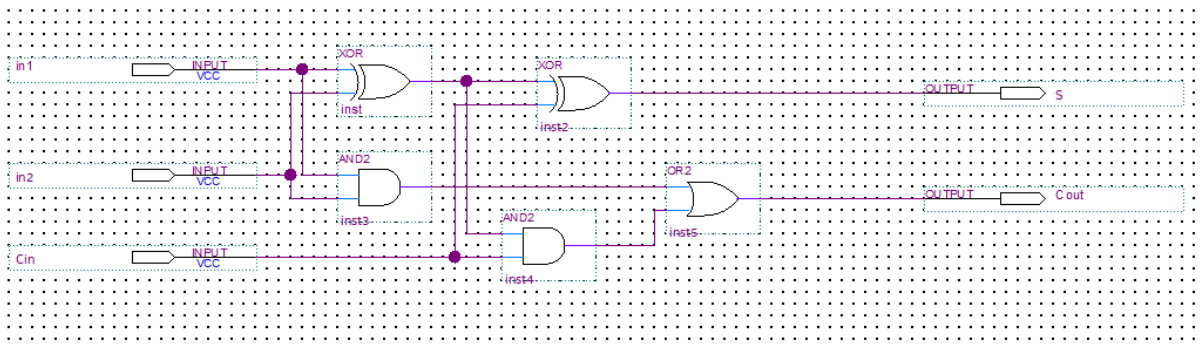
As previously stated, the **output_fsm** variable would be utilized later in **Part B** in determining which functionality of the calculator is being triggered, where it be the addition/subtraction or multiplication functionality

Four Bit Adder Subtractor

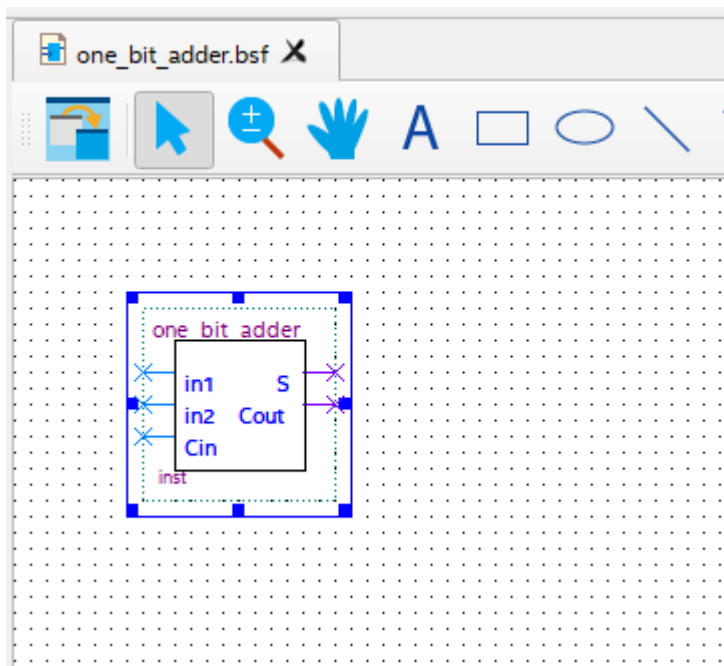


Above is a 4 bit adder and subtractor implemented together in a circuit, Much like what we worked on in previous labs, this adder looks similar in terms of concept, except this time rather than just adding two four bit numbers, we implemented an extra functionality to subtract those two four bit numbers as well. Using one bit adders that we created in a separate design file, we are able to add or subtract two four bits of number together as well as taking the carry in and carry out into account. This is further explained below.

One bit carry adder used:



One bit carry adder symbol:



Continuing on....

The functionality of the adder/sub module could be further explained in the event of subtracting or when FSM_0 is equal to 1 from the state logic of (FSM_1, FSM_0 = 0,1), we are utilizing 2's complement addition except that we would be adding a positive to a negative (negative represented by **b values**) in 2's complement.

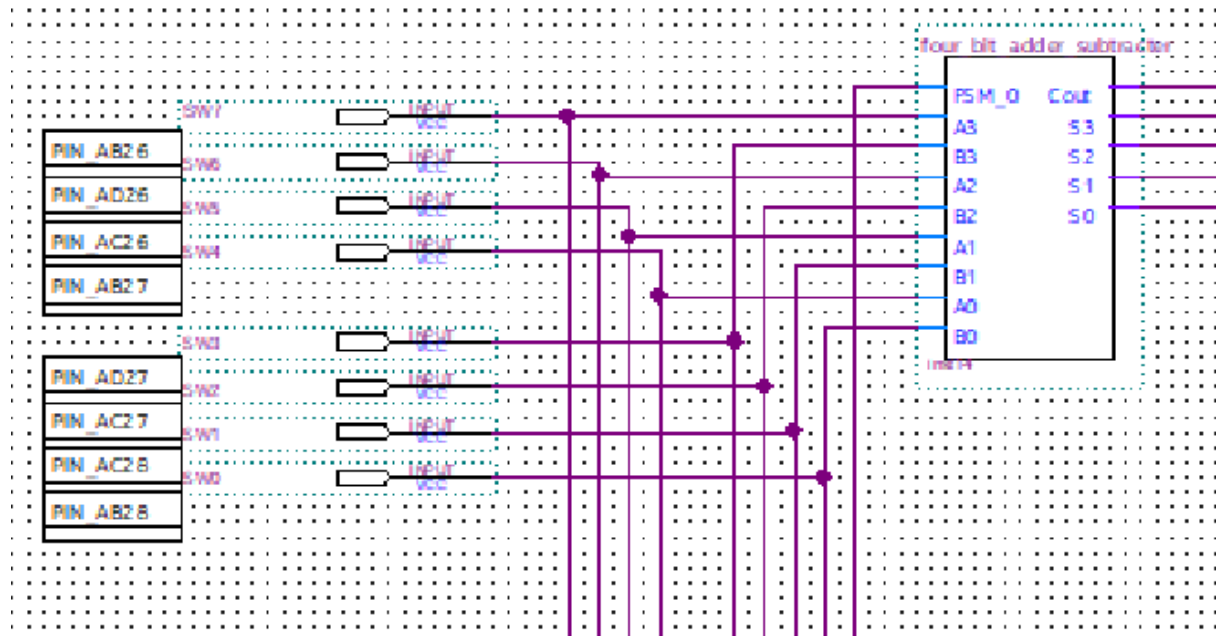
Therefore, we have to invert that negative-valued number into a positive by first converting it into 1's complement by inverting 1s to 0s and 0s into 1s, which can be done via an XOR gate connected to the input value. Next, we have to convert it back to 2' complement by using a carry-in of 1 which would be represented by our FSM_0 (1) value . Conversely, this would not happen if FSM_0 is equal to zero, indicating the addition function, therefore performing an addition of the inputted two four bit numbers. Below is an image representation of this logic being performed.

Subtraction logic being performed in the module:

Subtracting

$$\begin{array}{r} 1111 \\ - 1010 \\ \hline 1111 \\ 0101 \rightarrow \text{(1's complement)} \rightarrow \text{Inverting done with XOR gates} \\ + \quad (+1) \rightarrow \text{(2's complement converting)} \\ \hline 1111 \\ + 0110 \\ \hline \boxed{1} 0101 \\ \text{ignore.} \end{array}$$

The reason why we are taking 4 bits inputs to represent the number displayed on the hex display is because the functionality of it would require hexadecimals, and the best way to represent hexadecimals is parsing 4 bits to a seven segment decoder to the hex display.

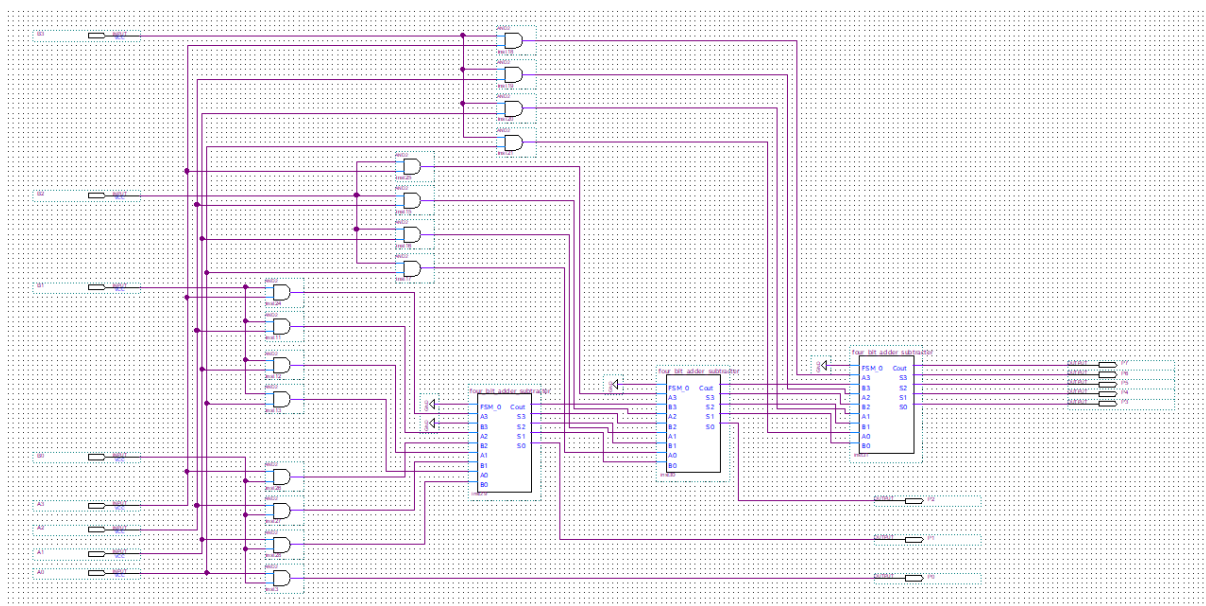


Above is the top level diagram of the 4 bit adder/subtractor with $a_3 - a_0$ representing the first number and $b_3 - b_0$ representing the second number to be multiplied together. These values are connected to 8 slide switches respectively. Within the previous labs, we have worked with slides switches being imputed to the adders so we decided to do the same. Again to reiterate, the FSM_0 input comes from the user (or w0 select line). Depending on the value of the FSM_0, where $FSM_0 = 1$, the module will perform subtraction and if it is a 0, it will perform addition.

Now, these output values from the adder/subtractor module would be later connected through a series of AND gates together with **output_fsm** which functions to only allow either the outputted added/subtracted or multiplied values to be stored in a register file. The way of determining which type of value should be stored on the register file is determined via the **output_fsm** previously mentioned in the FSM part, outputting a 1 when multiplied values should be allowed through and 0 for the added/subtracted values. This would be further explained later on in Part B.

Four Bit Multiplier

Much of the work below is the top level and inside view of the 4 bit multiplier. As we have stated in previous statements regarding the adder and subtractor, the inputs from this come from the user using the slides switches. The inputs are two 4 bit numbers in hexadecimal notation. For the output p7 is the most significant bit and p0 is the least. Utilizing three 4 bits adders/subtractor modules previously made, with their FSM_0 input being at constant zero value, the multiplier module performs basic hexadecimal multiplication on the inputs and outputs an 8 bits answer.



Here is a simple visualization of the multiplication logic being performed in the module.

Multiplication.

$$\begin{array}{r}
 1011 \\
 \times 1101 \\
 \hline
 1011 \\
 + 0000 \\
 + 1011 \\
 + 1011 \\
 \hline
 10001111
 \end{array}$$

MSB (p7) LSB (p0).

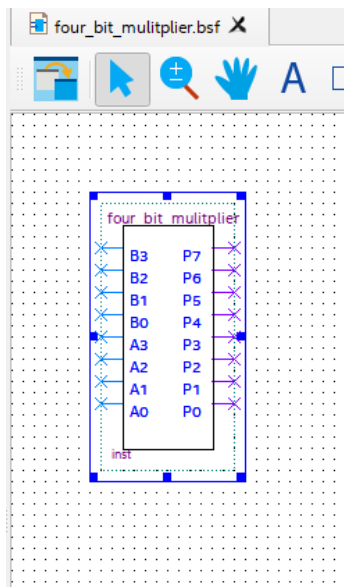
3 four bit adders → FCM-0 = 0 (must)
 so, connect to gnd.
 for adding function of
 adder/sub module.

$$\begin{array}{r}
 1011 \\
 \times 1 \\
 \hline
 1011
 \end{array}
 = (1011) \cdot (1)$$

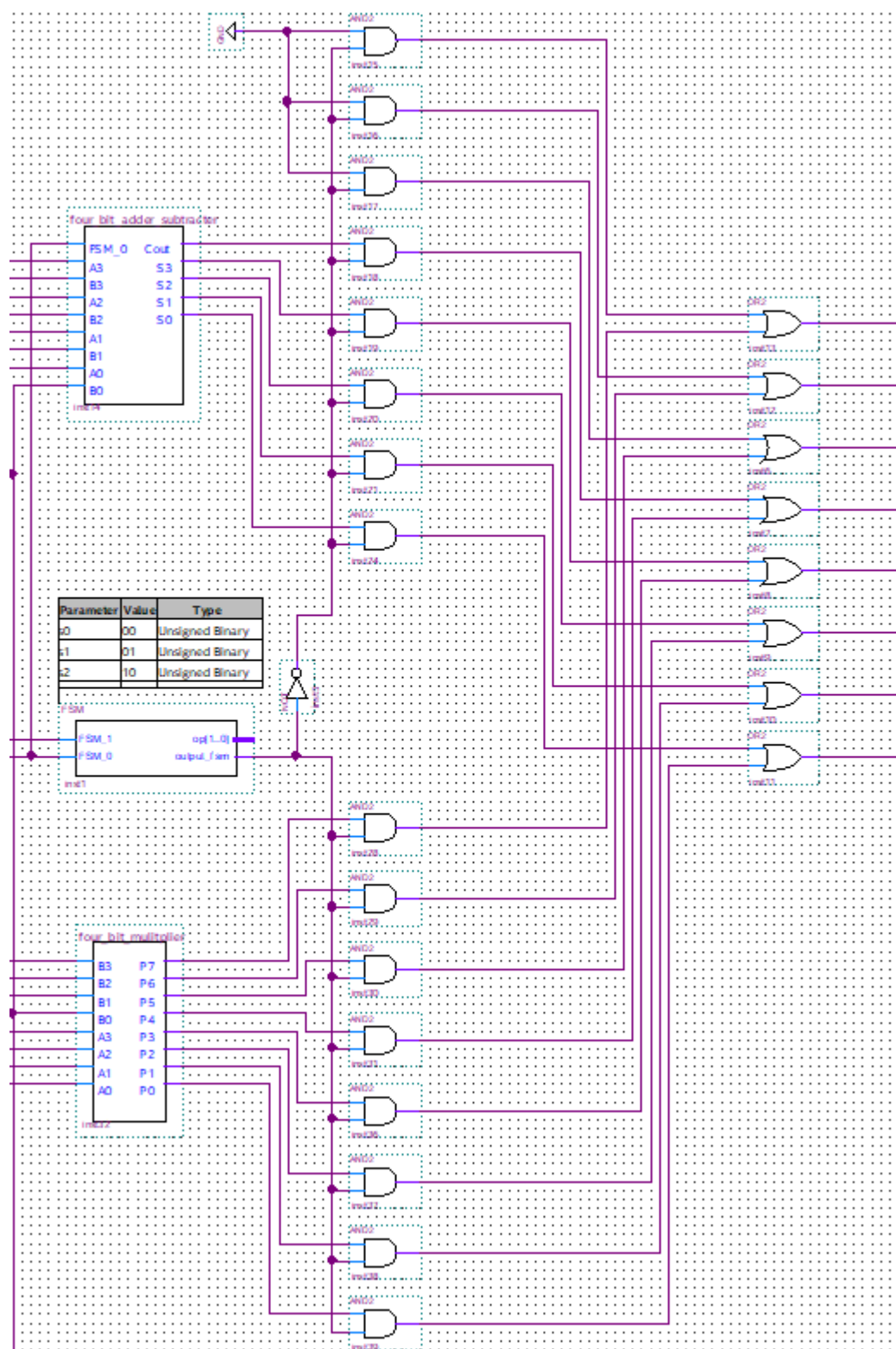
AND gates.

Scanned with CamScanner

This is the module for the multiplier.



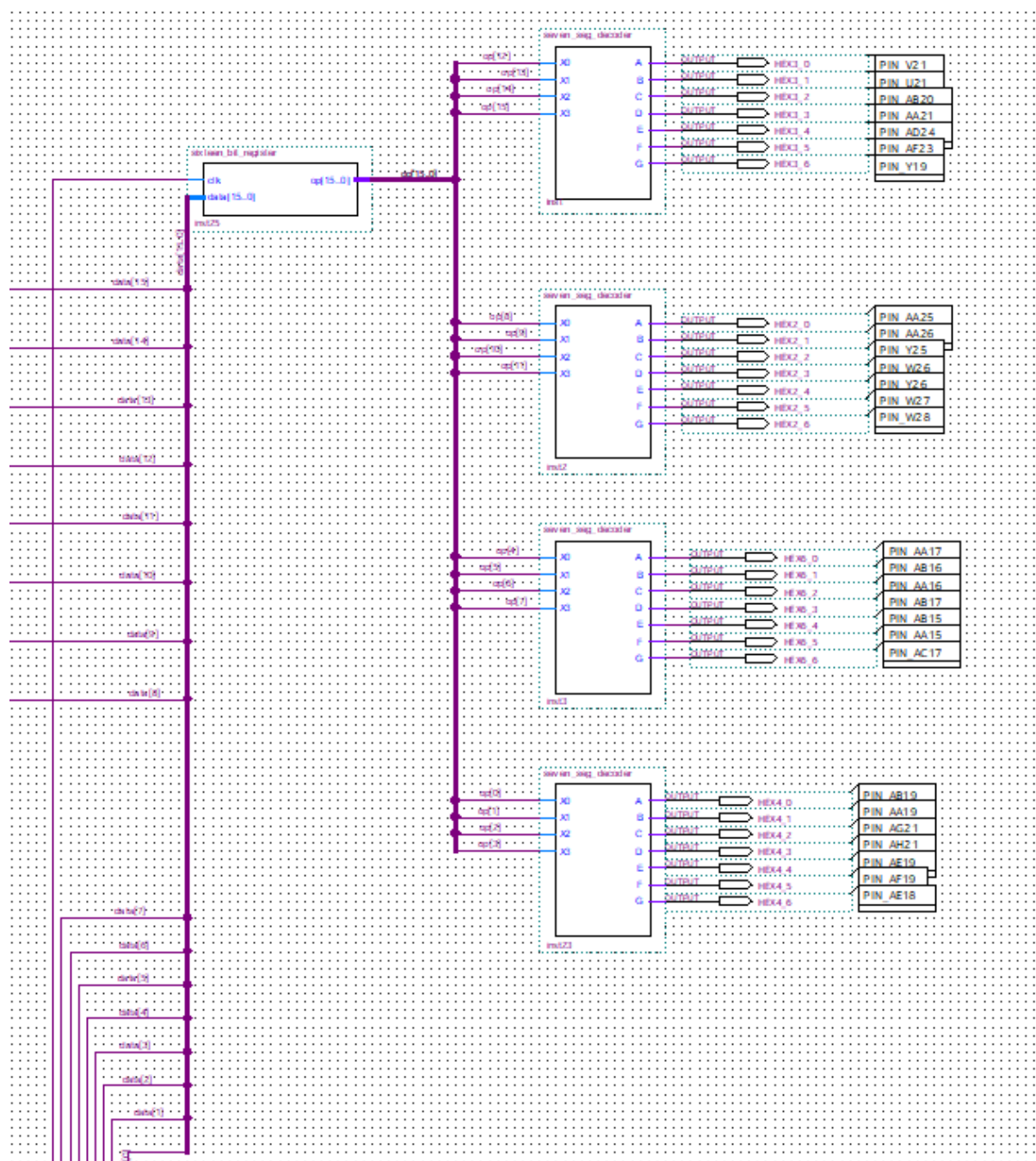
Part B



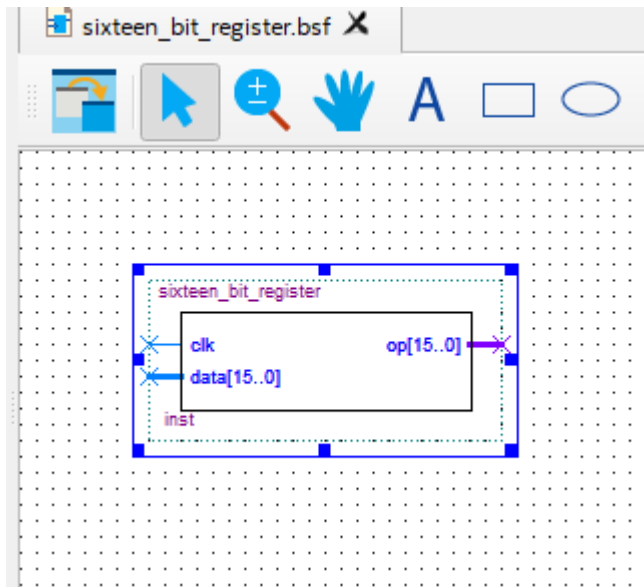
As previously mentioned, the output values from the adder/subtractor module and the multiplier module would be later connected through a series of AND gates together with **output_fsm** which functions to only allow either the outputted added/subtracted or multiplied values to be stored in a register file. The way of determining which type of value should be stored on the register file is determined via the **output_fsm** previously mentioned in the FSM part, outputting a 1 when multiplied values should be allowed through and 0 for the added/subtracted values. This value would be connected to a NOT gate triggering only one of the modules at a given time. Also, the reason why the outputs from the adder/sub module are connected to 8 AND gates instead of 6 AND gates would be because of the multiplication module having 8 outputs, therefore we had to smooth out the number of inputs going into the OR gates being 8 each from each module.

For simplification purposes, we will take the two outputs that went through the AND gates from the add/sub and multiplier module and connect them together with OR gates. This is done since either the outputted added/subtracted or multiplied values would be allowed through at a single time. This would in turn perfectly simplify the amount of inputs the register file would store later on in Part C, where the register file would store 8 bits of these calculated values (answer to this calculator) and 8 bits of the 2 numbers in hex that the user inputted through the slide switches, totaling to a 16 bit register.

Part C



16 bit Register



For this 16-bit register, we took inspiration from a similar design that we created in Lab 12, therefore coding this 16-bit register in Verilog. Using this 16-bit register, we are able to simulate an equal sign that is displayed on a real calculator. As previously mentioned, inputs of **data[15]** to **data[8]** of 8 bits would be delegated for the calculator's answer and the rest of the 8 bits of inputs being **data[7]** to **data[0]** are delegated for the 2 numbers in hex that the user inputted through the slide switches. The implemented code for this register file is given below. We also had to connect the register to a clock (clk) variable, being a press button on the FPGA board that acts as a manual clock. This will trigger the register to output data onto the seven segments display to display the two numbers in hex that the user inputted and the answer of the calculation onto a bus line of **op[15]** to **op[0]**, this is done to simulate an equal sign on real life calculators.

Verilog for the 16-bit register:

```
1  module sixteen_bit_register (  
2      input wire clk,  
3      input wire [15:0] data,  
4      output reg [15:0] op  
5  );  
6  
7      always @(posedge clk)  
8      begin  
9          op <= data;  
10     end  
11 endmodule  
12
```

Output

Through the seven segment display, we are able to output the data for the calculations much like we have done so in the labs. In order to get the right digits to display, we used 4 of the seven segment displays, 2 of them would display the numbers that the user chooses to input, and the other 2 outputs the answer due to it requiring 8 bits for the answer.

Above is the overall snippet of the seven segment display verilog code, as well as the connection of the 16 bit register connecting to it as well.

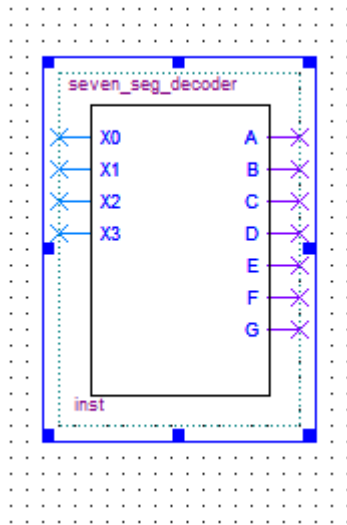
Seven Segment decoder

Below will contain the seven segment display, the top level view connection to the register as well as the functionality that we utilized from the previous labs. These seven segment decoders will be connected to a bus line of previously mentioned **op[15..0]** from the register's output.

Verilog for the seven segment decoder:

```
module seven_seg_decoder(x0,x1,x2,x3,A,B,C,D,E,F,G);
    input x0,x1,x2,x3;
    output reg A,B,C,D,E,F,G;
    always@(x0 or x1 or x2 or x3)
        case({x3,x2,x1,x0})
            4'b0000: {A,B,C,D,E,F,G}=7'b0000001;
            4'b0001: {A,B,C,D,E,F,G}=7'b1001111;
            4'b0010: {A,B,C,D,E,F,G}=7'b0010010;
            4'b0011: {A,B,C,D,E,F,G}=7'b0000110;
            4'b0100: {A,B,C,D,E,F,G}=7'b1001100;
            4'b0101: {A,B,C,D,E,F,G}=7'b0100100;
            4'b0110: {A,B,C,D,E,F,G}=7'b0100000;
            4'b0111: {A,B,C,D,E,F,G}=7'b0001111;
            4'b1000: {A,B,C,D,E,F,G}=7'b0000000;
            4'b1001: {A,B,C,D,E,F,G}=7'b0000100;
            4'b1010: {A,B,C,D,E,F,G}=7'b0001000;
            4'b1011: {A,B,C,D,E,F,G}=7'b1100000;
            4'b1100: {A,B,C,D,E,F,G}=7'b0110001;
            4'b1101: {A,B,C,D,E,F,G}=7'b1000010;
            4'b1110: {A,B,C,D,E,F,G}=7'b0110000;
            4'b1111: {A,B,C,D,E,F,G}=7'b0111000;
            default: {A,B,C,D,E,F,G}=7'b1111111;
        endcase
endmodule
```

Symbol for the seven segment decoder:



Conclusion

All in all, this final project made us realize the applicability of all the concepts that we learnt in class and how it correlates together in our final project of a hex calculator.